TrendLens

Algorithms & Pretrained Models

A detailed, code-grounded reference for the Review 2 walkthrough

Every model, every algorithm, and how CLIP works internally

Multimodal visual-trend detection for social media

Compiled from the project source (`src/`, `config.py`, model-selection result files)

Portable PDF · read offline

Contents

1. How to use this document
2. The system in one page
3. CLIP — the deep dive (your Review 1 answer)
    1. What CLIP is, and the problem it solves
    2. The two-tower architecture
    3. The image tower (ViT-B/32), step by step
    4. The text tower, step by step
    5. How it was trained: the contrastive objective
    6. How it is used at inference
    7. Why cosine similarity and L2 normalisation
    8. Zero-shot classification
    9. The modality gap (and why mean-centering matters)
    10. Where CLIP is used in TrendLens
    11. Ready-made oral answers (30 s / 2 min)
4. The pretrained models
    1. CLIP ViT-B/32 — image & text encoder
    2. BLIP large — the captioner / VLM
    3. BAAI/bge-base-en-v1.5 — the RAG text encoder
    4. The optional LLM writing layer
5. The algorithms
    1. L2 normalisation & cosine similarity
    2. UMAP — dimensionality reduction
    3. HDBSCAN — density clustering
    4. FAISS IndexFlatIP — vector search
    5. Stable cluster tracking (centroid lock + KNN)
    6. Zero-shot photography-style tagging
    7. BLIP caption aggregation
    8. The formal trend definition & the Poisson/binomial test
    9. RAG scope gating
    10. RAG retrieval & context assembly
    11. Evaluation metrics
6. Full pipeline walkthrough & artifacts
7. Data integrity & key design decisions
8. Likely review questions & model answers
9. Cheat sheet & glossary

1. How to use this document

This document is written so it can be read straight through, or jumped into at any heading. It is designed to answer two things:

1. **What is inside each pretrained model** — with CLIP treated in the most depth, because that was the question you could not answer in Review 1.
2. **What each algorithm does and why it was chosen** — including the evidence from the project's own model-selection runs.

> **The three things to be able to say out loud:**
>
> 1. TrendLens uses **CLIP** to turn both images and text into vectors in *one shared space*. Everything downstream (clustering, retrieval, style tags) is built on that one space.
> 2. It uses **BLIP** to caption representative images into words, **UMAP + HDBSCAN** to group visually-similar images without labels, and **FAISS** to retrieve the right group for a question.
> 3. The **LLM is only a writing layer** — it never invents facts; the product works with it switched off.

Conventions used below: file references appear like `src/embeddings.py:227`. Model names, thresholds and hyperparameters are quoted exactly as they appear in the source. Anywhere the project measured a choice, the measured numbers are given.

2. The system in one page

TrendLens is a **multimodal visual-trend detector**. It pulls real Instagram posts, groups them by what they *look* like (not by hashtags or captions), tracks each group over time, describes each group in words, and answers natural-language questions about what is trending and how to shoot it.

> **Research question:** can visual clusters derived from Instagram image embeddings reveal emerging patterns *before* textual labels appear? The project measures this with a "textual anonymity" metric — visual groups share far fewer hashtags than hashtag-based groups, which is direct evidence that the clusters capture patterns that have no name yet.

Data flow (end to end)

```
Instagram (Apify API)
      │  posts + image URLs
      ▼
download images ──────────────► data/instagram/images/*.jpg
      │
      ▼
CLIP image encoder  (ViT-B/32) ─► 512-d L2-normalised image vectors
      │
      ├─► baseline:  UMAP (512→15) ─► HDBSCAN ─► clusters (+ noise label -1)
      │
      ├─► incremental: FAISS KNN to locked centroids ─► stable cluster IDs
      │                                                  (+ micro-clusters)
      ▼
per cluster: representative images
      │
      ▼
BLIP captioning ─► deterministic aggregation ─► name / keywords / description
      │
      ├─► CLIP zero-shot style tags  (how it is SHOT)
      │
      ▼
temporal analysis ─► formal trend verdict (Rising / InsufficientData …)
      │
      ▼
RAG index: bge-base text embeddings + FAISS IndexFlatIP
      │
      ▼
user query ─► CLIP text embed ─► FAISS retrieve clusters ─►
              [ optional LLM rewrites retrieved evidence ] ─► answer
```

The eight pipeline phases

Phase	Module	What happens
1	`apify_client.py`	Fetch Instagram posts via the Apify REST API (last N days)
2	`data_collector.py`	Download images to `data/instagram/images/`
3	`data_collector.py`	**CLIP** ViT-B/32 image embeddings — 512-d, L2-normalised
4a	`data_collector.py`	**Baseline:** UMAP-15 + HDBSCAN clustering
4b	`cluster_tracker.py`	**Baseline:** lock centroids, assign stable UUIDs, build FAISS index
4c	`data_collector.py`	**Incremental:** FAISS KNN assignment to existing clusters
4d	`cluster_tracker.py`	**Incremental:** detect emerging micro-clusters from unassigned images
5	`data_collector.py`	**BLIP** captioning of representative images per cluster
6	`data_collector.py`	Temporal analysis: daily counts, growth rate, trend verdict
7	`data_collector.py`	FAISS RAG index over **bge** sentence-transformer embeddings
8	`rag.py`	Query → semantic retrieval → (optional LLM) answer

3. CLIP — the deep dive

This is the section to read carefully before Review 2. It explains what CLIP is, what is inside it, how it was trained, and how TrendLens uses it.

3.1 What CLIP is, and the problem it solves

CLIP stands for **C**ontrastive **L**anguage–**I**mage **P**re-training. It was released by OpenAI in 2021 (Radford et al., *Learning Transferable Visual Models From Natural Language Supervision*). The checkpoint TrendLens uses is `openai/clip-vit-base-patch32`.

The problem CLIP solves: normal image models are **classifiers**. A ResNet trained on 1,000 ImageNet classes can only ever output one of those 1,000 labels, and to teach it a new class you must retrain it. That is useless here, because a *visual trend* is an open-ended, unnamed category — "moody low-key food shot with harsh flash", say — that nobody labelled in advance.

CLIP solves this by learning a **shared embedding space** for images and text. After training, an image and a natural-language sentence are both mapped to vectors, and their similarity tells you how well they match. That gives three superpowers:

- **Open vocabulary.** You can describe a category in words at query time — no retraining.
- **Zero-shot.** You can classify or tag images using text prompts written *after* training.
- **Two modalities, one ruler.** Because text and images live in the same space, a text query can retrieve images, and an image can retrieve text.

> **One-sentence definition.** CLIP is a *dual-encoder* (two-tower) neural network, trained contrastively on ~400 million image–caption pairs, that projects images and text into one shared ~512-dimensional space where a matching image and caption have high cosine similarity.

3.2 The two-tower architecture

CLIP has **two separate encoders** that never see each other's input:

Tower	Input	Architecture	Output
Image encoder	A 224×224 RGB image	Vision Transformer, "ViT-B/32" (Base size, patch 32)	A 512-d vector
Text encoder	A tokenised caption	A Transformer (12 layers, 512 width)	A 512-d vector

Each tower ends with a small learned **linear projection** that maps its internal representation into the *same* 512-d space. In TrendLens this dimension is read from the model rather than hard-coded:

```
# src/embeddings.py:324
def proj_dim(self):
    if self._proj_dim is not None:
        return int(self._proj_dim)
    if getattr(self.model, "config", None) is not None:
        return int(self.model.config.projection_dim)   # 512 for ViT-B/32
    ...
```

Important subtlety: the two towers are **not** joined into one network. There is no shared layer that mixes an image and a sentence. Each side is encoded independently, and the only thing that connects them is the *training objective* that pushed matching pairs together in the shared space. This is what makes inference cheap: you can pre-compute all image vectors once, then compare any text against them with a dot product.

3.3 The image tower (ViT-B/32), step by step

"ViT-B/32" decodes as: **V**ision **T**ransformer, **B**ase size, patch **32**. Here is the exact sequence for a 224×224 input:

1. **Split into patches.** The image is cut into non-overlapping squares of 32×32 pixels. 224 ÷ 32 = 7, so there is a 7×7 grid = **49 patches**. Each patch is 32×32×3 = 3,072 raw numbers.
2. **Flatten and project each patch.** Each 3,072-number patch is flattened and multiplied by a learned matrix that projects it to the model width (768 for the Base model). Now each patch is a 768-d token.
3. **Add a `[CLS]` token.** A special learned "classification" embedding is prepended. The sequence is now 1 + 49 = **50 tokens**.
4. **Add positional embeddings.** A learned vector for each position is added, so the model knows where each patch sat in the grid.
5. **Transformer stack.** The 50 tokens pass through **12 Transformer blocks**. Each block applies multi-head self-attention (every patch can look at every other patch) followed by a small MLP, with layer-norm and residual connections. Hidden width 768, 12 attention heads.
6. **Read the `[CLS]` output.** After the last block, the final layer-norm is applied and the **`[CLS]` token's vector** is taken as the image's global representation. (It is not the average of patches; the `[CLS]` token is trained to aggregate the whole image.)
7. **Project into the joint space.** A final learned linear layer maps the 768-d `[CLS]` vector to a **512-d** embedding.

Why a Transformer and not a CNN? A convolutional network builds up local features layer by layer. A ViT treats the image as a sequence of patches and lets *self-attention* relate any patch to any other patch from the very first layer. For trend detection this matters: whether a photo is a "top-down flat lay" or a "45-degree diner's angle" is a *global arrangement* property, not a local texture, and self-attention captures global arrangement directly.

In TrendLens, the image vector is produced by `get_image_features` and then L2-normalised:

```
# src/embeddings.py:227
with torch.no_grad():
    inputs = self.processor(images=images, return_tensors="pt")
    inputs = {k: v.to(self.device) for k, v in inputs.items()}
    outputs = self.model.get_image_features(**inputs)
    feats = outputs.pooler_output if hasattr(outputs, "pooler_output") else outputs
feats = feats.float().cpu().numpy()
return l2_normalize(feats)          # unit-length vector
```

3.4 The text tower, step by step

1. **Tokenise.** The caption is split into sub-word tokens using a byte-pair-encoding (BPE) vocabulary of ~49,408 tokens. A caption is wrapped with start-of-text and end-of-text markers and padded/truncated to at most 77 tokens.
2. **Embed tokens.** Each token becomes a learned vector, then a positional embedding is added.
3. **Causal Transformer.** The tokens pass through 12 Transformer layers with masked (causal) self-attention, width 512.
4. **Read the end-of-text token.** The final hidden state of the **end-of-text** token is taken as the sentence's representation (it has "seen" the whole caption).
5. **Project.** A learned linear layer maps it to the same **512-d** joint space as the image tower.

TrendLens uses the text tower in several places, always through the same helper:

```
# src/retrieval.py:57
def embed_texts(model, processor, texts, device=None, batch_size=16):
    ...
    out = model.get_text_features(**inputs)
    out = out.pooler_output if hasattr(out, "pooler_output") else out
    out = out.detach().cpu().numpy()
    out = out / (np.linalg.norm(out, axis=1, keepdims=True) + 1e-12)
    return out.astype("float32")        # L2-normalised text vectors
```

> **Same model, both sides.** The project deliberately uses *one* CLIP checkpoint for image embeddings, query embeddings, style-tag prompts, and scope-gating anchors. This is the single most important architectural fact: image and text vectors are guaranteed to live in the same space, so comparing them is always valid. `config.py:219` states this explicitly.

3.5 How it was trained: the contrastive objective

CLIP is trained on roughly **400 million (image, caption) pairs** scraped from the web. There are no manual labels — the caption *is* the supervision. Training works on batches:

1. Take a batch of **N** image–caption pairs (N is large, e.g. 32k).
2. Encode all N images and all N captions → two sets of N vectors in the shared space.
3. Compute the **N×N matrix of cosine similarities**: every image against every caption.

4. The **diagonal** entries are the correct (matching) pairs. The **N²−N off-diagonal** entries are mismatches.
5. Apply a **symmetric cross-entropy loss** (a form of InfoNCE): for each image, maximise the probability of its own caption and minimise the probability of all others, and symmetrically for each caption. A learned temperature parameter scales the similarities.

So the objective is literally: **pull each image and its own caption together, and push it away from every other caption in the batch.** Repeat over 400M pairs and the geometry of the space organises itself so that visual and semantic similarity line up.

> **Why "contrastive"?** The model learns by *contrast* — what makes a matching pair different from a mismatched pair. It never predicts a label, so it is never limited to a fixed label set. That is exactly why it can tag styles nobody named during training.

3.6 How it is used at inference

At inference there is **no training and no label mapping**. It is just two forward passes and a dot product:

```
# Conceptually, for a text query against pre-computed image vectors:
img_vec = encode_image(image)          # (512,), L2-normalised
txt_vec = encode_text("a moody food photo lit by harsh flash")  # (512,)
similarity = img_vec @ txt_vec         # cosine similarity, in [-1, 1]
```

Because the vector spaces are fixed, TrendLens pre-computes all image vectors once (Phase 3) and caches them; every later comparison is a matrix product. Style tagging is literally one line:

```
# src/style_tags.py:579
sims = img @ text_embs.T               # (N images, 30 style prompts)
```

3.7 Why cosine similarity and L2 normalisation

Cosine similarity measures the **angle** between two vectors and ignores their lengths. Two vectors pointing the same way score 1.0 whether they are long or short. If both vectors are forced to unit length (|v| = 1) by **L2 normalisation**, then:

> **cosine(a, b) = a · b** — the angle becomes a simple dot product. This is why the entire codebase normalises every embedding to unit length and then uses dot products everywhere. It also means the FAISS index can be an *inner-product* index (`IndexFlatIP`) and still implement cosine similarity.

TrendLens enforces the property with an assertion rather than trusting it:

```
# src/embeddings.py:94
def assert_alignment(embeddings, df):
    assert embeddings.dtype == np.float32
```

```
    norms = np.linalg.norm(embeddings, axis=1)
    assert np.allclose(norms, 1.0, atol=1e-4)   # every row is unit length
```

3.8 Zero-shot classification

"Zero-shot" means classifying into categories that were never seen in training. CLIP does it by turning each label into a sentence:

```
labels  = ["a photo of a dog", "a photo of a cat", "a photo of a car"]
# encode all labels with the text tower -> 3 vectors
# encode the image -> 1 vector
# the predicted label = argmax cosine similarity
```

TrendLens uses this idea for **photography style tags** (`src/style_tags.py`). Instead of subject labels ("dog"), it uses execution labels ("a photo lit by soft daylight coming from a window"), and each tag carries a small **prompt ensemble** — several phrasings averaged together to reduce the noise of any one wording:

```
{
  "tag": "natural window light",
  "aspect": "lighting",
  "prompts": [
    "a photo lit by soft daylight coming from a window",
    "a naturally side-lit photo with gentle soft shadows",
  ],
  "direction": "use soft natural window light from the side",
}
```

There are **15 tags across 5 aspects** (framing, lighting, color mood, composition, process), with **30 prompts** total. Per tag, the project averages its prompt similarities; per cluster, it averages the per-image tag scores.

3.9 The modality gap (and why mean-centering matters)

This is a subtle but senior-level point, and the project found a concrete bug from ignoring it.

CLIP image vectors and text vectors, even when matching, sit in slightly **separate regions** of the shared space — the "modality gap". Consequence: all raw cosine similarities between an image and any text prompt fall in a narrow band (roughly 0.15–0.30). The *absolute* value says little; what matters is the *relative* ordering.

> **The failure the project observed.** With raw scores, the global per-tag mean across all 15 tags spanned only 0.1715–0.2082, and the same two tags — "eye-level straight-on" and "natural window light" — won on *8 of 12* clusters. That is not tagging the cluster; it is describing Instagram photographs in general.

The fix (`src/style_tags.py:394`): subtract each image's own mean across all tags. A score then means "how much more this tag fits this image than the average style description fits it". This removes the modality gap and makes clusters separable.

```
# src/style_tags.py:418
def scores_from_prompt_sims(prompt_sims):
    out[:, ti] = sims[:, mask].mean(axis=1)     # mean over each tag's prompts
    return out - out.mean(axis=1, keepdims=True) # subtract the image's own mean
```

Those centered margins are then mapped to a 0–1 presentation scale, and a tag must clear `MIN_STYLE_SCORE = 0.55` (55% of the strongest available signal) to be reported at all.

3.10 Where CLIP is used in TrendLens

Use	Which tower	File
Image embeddings for clustering	Image tower	`src/embeddings.py`
Query embedding for retrieval	Text tower	`src/retrieval.py`, `src/rag.py`
Cluster text embeddings for the index	Text tower	`src/retrieval.py`
Zero-shot photography-style tags	Image × Text	`src/style_tags.py`
Scope-gating anchors ("in scope vs out of scope")	Text tower	`src/rag.py`
CLIPScore caption-quality metric	Image × Text	`scripts/select_caption_model.py`

3.11 Ready-made oral answers

The 30-second answer

> "CLIP is a dual-encoder model trained contrastively on 400 million image–caption pairs. One tower is a Vision Transformer that encodes an image; the other is a text Transformer that encodes a caption. Both project into the same 512-dimensional space, and training pulls a matching pair together while pushing it away from the other captions in the batch. At inference I encode each side separately and take the cosine similarity — a dot product, because both are L2-normalised. That shared space is what lets my code compare images to a text query, tag styles with text prompts, and retrieve clusters."

The 2-minute answer (add structure)

> "There are four things to say. **First, the shape:** two independent towers — a ViT-B/32 image encoder and a text Transformer — each ending in a linear projection into one 512-d shared space. **Second, the image path:** 224×224 is cut into 32×32 patches, so 49 patches plus a `[CLS]` token; those 50 tokens pass through 12 Transformer blocks and the `[CLS]` output is projected to 512-d. **Third, training:** a symmetric contrastive loss on the N×N image–text similarity matrix of each batch pulls the diagonal (true pairs) together and pushes off-diagonal (false pairs) apart. **Fourth, use:** it is not generative — it only produces vectors. Because image and text vectors share one space and are unit-length, inference is just a dot product, which is how one model powers clustering, retrieval and zero-shot style tagging."

4. The pretrained models

TrendLens uses exactly three pretrained models in its core pipeline plus one optional writing model. Crucially, every model name is declared *once* in `config.py` and nowhere else, so a swap cannot leave a stale hard-coded copy behind (a test enforces this — `tests/test_cache_invalidation.py`).

4.1 CLIP ViT-B/32 image + text

Field	Value
Checkpoint	`openai/clip-vit-base-patch32`
Declared at	`config.py:230` (`CLIP_MODEL`)
Embedding dim	512 (read from `config.projection_dim`)
Input image	224×224 (native for every candidate evaluated)
Roles	Image embeddings · query text embeddings · style tags · scope anchors

Why this one? It was chosen by measurement, not by default. `scripts/select_models.py` evaluated **5 encoders × 18 clustering configurations** and ranked by mean Spearman ρ over by-account splits, behind coverage / cluster-count / stability gates. From `model_selection_summary.json`:

Encoder	Selection ρ	Stability ARI	Coverage
openai/clip-vit-base-patch32	**0.166**	**0.726**	0.822
openai/clip-vit-large-patch14	0.163	0.771	0.859
google/siglip2-base-patch16-224	0.141	0.562	0.794
openai/clip-vit-base-patch16	0.122	0.570	0.690
laion/CLIP-ViT-B-32-laion2B	0.068	0.665	0.690

> **The reasoning to quote:** ViT-B/32 wins on the primary metric *and* on cluster stability. Larger ViT-L/14 is statistically indistinguishable on ρ but costs ~3× the encode time, so it was not adopted. The newer SigLIP2 was explicitly tested and did **not** displace CLIP (lower ρ, much lower stability, ~2× the time) — the newer model family is not automatically better here.

4.2 BLIP large captioner / VLM

Field	Value
Checkpoint	`Salesforce/blip-image-captioning-large`
Declared at	`config.py:265` (`BLIP_MODEL`)
Type	Vision-Language Model: a ViT-L/16 image encoder + an autoregressive text decoder
Role	Caption representative images per cluster → cluster names, keywords, RAG text
Loaded / run in	`src/interpretation.py:59` (`load_blip`, `caption_image`)

What BLIP does here. For each cluster, up to `k=4` representative images are captioned ("a wooden table with a brunch spread"). Captions are then aggregated deterministically into a cluster name, description and keyword list.

Why "large" and not "base"? Measured on 60 real corpus images (`caption_model_selection_results.csv`):

Model	CLIPScore	distinct-2	Degenerate	Speed
blip-image-captioning-base	0.2740	0.704	5%	~0.53 img/s
blip-image-captioning-large	**0.2775**	**0.923**	**0%**	~0.04 img/s

The CLIPScore gap is within noise at n=60, so the deciding reasons are **lexical diversity** (cluster names stop repeating each other in the UI) and the **absence of degenerate repetitions**. The cost is accepted knowingly: ~0.04 img/s means interpreting ~19 clusters takes roughly half an hour instead of ~2.5 minutes.

> **Integrity note to state clearly:** BLIP captions and the names derived from them are **model interpretations, not ground truth**. They are labelled as such in `cluster_captions.json` and the report (`src/interpretation.py:12-18`).

4.3 BAAI/bge-base-en-v1.5 RAG text encoder

Field	Value
Checkpoint	`BAAI/bge-base-en-v1.5`
Declared at	`config.py:247` (`RAG_EMBED_MODEL`)
Type	Sentence-Transformer (BERT-base sized, ~110M params), 768-d output
Role	Embeds RAG text chunks and user queries for the FAISS text index
Used in	`src/rag.py`, `src/data_collector.py` (`build_rag_index`)

Why a separate model from CLIP? This encoder only ever compares **text to text** (chunk text to query text). The strongest general text-retrieval checkpoint is not the strongest image–text one, so the project decoupled them deliberately.

Why bge and not the sentence-transformers default? Chosen by `scripts/select_text_model.py` on the project's own retrieval task (BLIP captions of cluster members as

queries, the representative excluded so no query is a copy of indexed text). Over 6 candidates bge-base won on every metric:

Model	R@1	R@3	MRR
all-MiniLM-L6-v2 (incumbent)	0.347	0.542	0.502
BAAI/bge-base-en-v1.5	**0.431**	**0.694**	**0.590**

Detail worth knowing: bge models need no query/passage prefix (that is an e5-family convention, handled explicitly in the selection script).

4.4 The optional LLM writing layer optional

Field	Value
Providers	Gemini · OpenAI · Ollama (local)
Default models	`gemini-3.1-flash-lite` · `gpt-4o-mini` · `llama3.2`
Declared / run in	`src/llm.py`
Temperature	0.3 (low on purpose — faithful rewriting, not creative generation)
Enabled by	`TRENDLENS_LLM_PROVIDER` (unset = disabled)

> **The rule that governs the LLM:** it is a **writing layer only**. It rewrites already-retrieved, measured evidence into plain-language prose. It is *never* a knowledge source and is forbidden from inventing photography advice. Every instruction it may give traces to a *measured* style tag. On any failure — missing key, network error, bad response — the function returns `None` and the caller transparently falls back to the deterministic formatter (`src/llm.py:19-21, 314-387`).

Three defences make "the LLM cannot fabricate" a design property, not a hope:

1. **Structured evidence only.** The model receives a compact JSON bundle of measured facts (name, description, BLIP caption, *authored* shot recipe, median engagement). It never receives raw cluster IDs or internal metrics it could quote (`_trim_evidence`).
2. **Authored recipes.** The tag→steps mapping lives in code (`SHOT_STEPS`), so a step can only exist behind a tag CLIP actually measured. The model selects and sequences; it does not invent (`src/style_tags.py:205-220`).
3. **System-prompt prohibitions.** No cluster IDs, no pipeline internals (CLIP, HDBSCAN, UMAP, FAISS, RAG), no invented hashtags/cameras/lenses, no promised like counts. Median engagement only — never the mean (a single viral post moves the mean by orders of magnitude; observed max 15.5M likes against a 7k median).

5. The algorithms

5.1 L2 normalisation & cosine similarity

What: scale every embedding to unit length; then compare vectors by their dot product.

Why: cosine similarity is the natural metric for CLIP embeddings (training used it), and normalising to unit length turns it into a plain dot product — fast and index-friendly (see 3.7). **Where:** universally — images, text queries, style prompts, scope anchors. `float32` is enforced to match CLIP's precision and halve memory versus float64 (`src/embeddings.py:86-109`).

5.2 UMAP — dimensionality reduction

Field	Value
Full name	Uniform Manifold Approximation and Projection
Role	Reduce 512-d CLIP vectors to a low-dimensional manifold before clustering
Production params	`n_components=15`, `n_neighbors=15`, `min_dist=0.0`, `metric="cosine"`, seeded
File	`src/clustering.py:60` (`reduce_dimensions`)

How it works (intuition + mechanics): UMAP builds a **fuzzy weighted graph** of the high-dimensional data (each point connected to its near neighbours), then finds a low-dimensional layout that best preserves that graph by optimising a cross-entropy between the high- and low-dimensional graphs. `n_neighbors` controls the local-vs-global balance; `min_dist` controls how tightly points may pack. `min_dist=0.0` lets same-cluster points clump, which is what density clustering wants.

Why not cluster on the raw 512-d vectors? In high dimensions, distances concentrate and density-based clustering degrades. UMAP first denoises and exposes the local manifold structure, so HDBSCAN can find real groups. The project still compares raw-vs-UMAP honestly in its reports.

Choosing 15 dimensions — real evidence. Two sweeps disagreed, so a proper paired re-measurement over **80 held-out account splits** settled it (`config.py:309-322`):

```
UMAP  10   rho 0.170 ± 0.132
UMAP  15   rho 0.248 ± 0.113   <- production
UMAP  20   rho 0.177 ± 0.118

15 vs 20: +0.071, better on 65/80 splits, paired t = 6.96, p = 9e-10
```

The nominally "auto-selected" 20-d config was actually worse. This is a good story to tell: the automated selector averaged too few splits and was wrong; the corrected measurement won.

Determinism and caching: UMAP is seeded for reproducibility, and results are cached to `.npy` keyed on a **content fingerprint** of the input matrix. This exists because row-count

alone is not a valid cache key: every encoder produces the same number of rows for the same corpus, so a stale projection could silently survive an encoder swap (`src/clustering.py:33-57`).

5.3 HDBSCAN — density clustering

Field	Value
Full name	Hierarchical Density-Based Spatial Clustering of Applications with Noise
Role	Group visually similar images; label outliers as noise
Production params	`min_cluster_size=5`, `min_samples=3`, `cluster_selection_method="eom"`, Euclidean, `prediction_data=True`
File	`src/clustering.py:136` (`run_hdbscan`)

How it works: HDBSCAN converts distances into a **mutual-reachability** metric (a point must be close to its neighbour *and* that neighbour close to it), builds a minimum spanning tree, and derives a full **cluster hierarchy**. It then extracts the most **stable** clusters (the "excess of mass" / EOM criterion) rather than committing to one global density threshold.

Why HDBSCAN over the alternatives:

- **vs KMeans:** KMeans forces every point into one of k clusters and needs k up front. Visual trends have an unknown number of groups and a genuine "none of the above" (blurry, off-topic, one-off images).
- **vs DBSCAN:** DBSCAN needs a single global epsilon that rarely fits all clusters; HDBSCAN handles **varying densities** automatically.
- **Noise as a first-class output:** outliers get label `-1` and are never forced into a cluster. The project reports them honestly (e.g. 82 noise posts of 461).
- **New-point support:** `prediction_data=True` enables assigning future points consistently, which the incremental path relies on.

Reported diagnostics (`src/clustering.py:190`): cluster sizes, noise %, per-cluster membership probabilities, **cluster persistence**, and **silhouette score**. Representative images are ranked by membership probability, then distance to the cluster medoid.

5.4 FAISS IndexFlatIP — vector search

Field	Value
Library	FAISS (`faiss-cpu`), Facebook AI Similarity Search
Index type	`IndexFlatIP` — exact, brute-force inner-product search
Role	Retrieve nearest cluster-text / RAG-chunk vectors for a query
File	`src/retrieval.py:158` (`build_index`, `query_index`)

How it works: `IndexFlatIP` stores every vector and, at query time, computes the inner product against all of them, returning the top-k. Because stored and query vectors are L2-normalised, inner product *is* cosine similarity. It is exact (no approximation), which is

appropriate: the corpus is small (dozens to thousands of vectors), so the speed of an approximate index (IVF/HNSW) is unnecessary and would cost recall.

5.5 Stable cluster tracking — centroid lock + FAISS KNN + micro-HDBSCAN

Field	Value
File	`src/cluster_tracker.py`
Assignment threshold	0.25 cosine similarity
Micro-cluster trigger	≥ 3 unassigned images

The problem it solves: re-clustering from scratch every run reshuffles cluster IDs, which destroys time-series history. A cluster must keep the same identity across runs if you ever want to say "this group is growing".

The algorithm:

```
Baseline run:
  HDBSCAN -> labels -> compute centroids -> LOCK into registry (stable UUIDs, e.g.
cls_a1b2c3d4)

Incremental run:
  new images -> CLIP embed -> FAISS KNN against locked centroids
      similarity >= 0.25  -> assign to existing cluster
      similarity <  0.25  -> add to pending candidates
          candidates >= 3 -> HDBSCAN micro-cluster -> mint NEW stable IDs
```

Why this matters: it is the mechanism behind two README claims — "stable cluster IDs" and "centroid locking" — and it lets new visual trends be *born* as micro-clusters rather than being forced into existing ones.

5.6 Zero-shot photography-style tagging

Covered in depth in section 3.8–3.9. Summary of the algorithm:

1. Define a 15-tag taxonomy over 5 aspects (framing, lighting, color mood, composition, process).
2. Encode each tag's prompt ensemble with CLIP's text tower → (30, 512) matrix.
3. Compute `sims = image_embeddings @ text_embs.T` → (N, 30).
4. Average each tag's prompts → (N, 15); subtract each image's own mean across tags (remove modality gap).
5. Map to 0–1; per cluster, average member scores and keep tags above `MIN_STYLE_SCORE=0.55`.
6. Build an ordered **shot recipe** from authored `SHOT_STEPS`, ordered lighting → framing → colour → composition → process.

Why this exists: BLIP captions describe the *subject* ("a wooden table with a brunch spread") but not the *execution* (lighting, angle, colour grade). Without style tags, "what food-photography styles are trending?" degenerates into a list of what people are shooting instead of how. Style tags are computed on the *already-stored* image embeddings — no extra model pass.

5.7 BLIP caption aggregation

Field	Value
File	`src/interpretation.py` (`interpret_cluster`)
Input	Up to 4 BLIP captions per cluster
Output	name, description, characteristics, confidence, raw captions

1. **Tokenise & clean:** lowercase, strip stop-words and short tokens.
2. **Count** unigrams and bigrams across the cluster's captions.
3. **Name** = most common bigram (fallback: top unigram). **Description** = most repeated caption, or a keyword sentence. **Characteristics** = top unigrams.
4. **Confidence** = `1 − normalised Shannon entropy` over the unigram distribution — how much the captions agree with each other (1 = all identical, 0 = maximally dispersed).

This is fully deterministic given the captions, which is important: the only non-deterministic part is BLIP's generation, and that is labelled an interpretation.

5.8 The formal trend definition & the Poisson/binomial test

Field	Value
File	`src/trend_definition.py`
Windows	recent = 4 days, prior = 4 days
Gates	G1 support ≥ 8 · G2 persistence ≥ 3 active days · G2b ≥ 3 accounts · G3 growth ≥ +50%/day · G4 significance · G5 engagement floor · G6 coverage ≥ 60%
Verdicts	Rising · Falling · Flat · **InsufficientData**

The problem it fixes. Earlier, three different "growth" formulas coexisted (a raw count divided by log, a simple recent/prior ratio, and a README "+200%" claim), none comparable. On the real corpus 87% of posts fell in a single 11-day window, so a recent-vs-prior comparison was comparing 2 posts against 3 — pure sampling noise.

The definition: a cluster is a **rising trend** only if it passes **all** gates simultaneously; otherwise it is reported as "insufficient data" rather than silently ranked.

The statistical test. Gate G4 is an exact **one-sided binomial test**. Under the null of no growth, both windows share one underlying rate, so the recent window is a binomial draw with `p = n_prior / (n_prior + n_recent)`; the p-value is the probability of seeing at least the observed recent count in the upward tail:

```
# src/trend_definition.py:167  P(X >= n_recent) under Binomial(total, p_null)
p_value = sum(comb(total,k) * p_null**k * (1-p_null)**(total-k)
              for k in range(n_recent, total+1))
```

Why this matters: a raw percentage is not evidence. "0 → 4 posts is +400%" and carries no statistical weight; the binomial test stops that being reported as a trend. Both growth notions are reported separately and never conflated: **post-volume growth** (are there more posts?) vs **engagement growth** (do the same posts earn more likes?) — supply versus demand.

> **"InsufficientData" is a feature, not a failure.** On a single-snapshot corpus most clusters land there, and that is the *correct* answer. The point of the strict definition is to make the system say "I do not know" instead of emitting a confident number it cannot support.

Longitudinal store. A SQLite database (`TrendHistory`) appends one row per cluster per snapshot, so trend claims become testable over time. It refuses to treat a re-run over an *unchanged* corpus as new history, using a corpus fingerprint of the post IDs — because a re-clustering after a config edit is a *recomputation*, not the passage of time (`src/trend_definition.py:461-477`).

5.9 RAG scope gating

Goal: refuse questions outside the visual-trend domain (code, maths, recipes, news) instead of answering them with irrelevant trends.

Two stages (`src/rag.py:846`, `classify_scope`):

1. **Decisive keyword patterns.** A bank of regex patterns for unmistakable out-of-scope requests (programming, maths, cooking, science, news/sports/markets, translation, medical/legal/financial advice, homework). A full-phrase match is an immediate veto — a single token like "code" never vetoes a genuine visual question.
2. **Semantic anchor test.** If no keyword veto fires, embed the query with CLIP and compare its maximum cosine similarity against two curated anchor banks — in-scope questions ("coffee latte art photography", "flower photography", …) and out-of-scope questions ("write a python program", "how long to bake bread", …). In-scope iff:

   ```
   domain_score >= 0.22  AND  (domain_score - out_score) >= -0.01
   ```

Anchors are cached and keyed on their exact text. If the model/index is unavailable, the gate **fails open** (`method="fallback"`), so a missing artifact degrades to no gating rather than blocking all queries.

Topic gating is a separate mechanism used by the deterministic fallback formatter: a curated **topic lexicon** plus stemming and substring matching decides whether the corpus actually covers the question's subject. If a subject is named but nothing covers it, the system returns an explicit **no-coverage** answer naming what it does cover — instead of answering a makeup question with fried chicken.

5.10 RAG retrieval & context assembly

The query path (`src/rag.py:938`, `build_context`):

1. Embed the query with CLIP's text tower.
2. Search the FAISS cluster-text index → top-k clusters (k=5; chosen at the knee of the measured hit-rate/MRR curve).
3. Assemble honest records: name, description, sample caption, style tags, engagement, representative image — with unmeasured fields emitted as `null`/omitted, never invented.
4. Optionally, retrieve supporting text chunks from the **bge** RAG index and hand the bundle to the LLM writing layer.

> **Two retrieval indexes, two encoders, on purpose:** the *cluster* index uses CLIP text vectors so a query lands in the same space as the image clusters; the *RAG chunk* index uses bge vectors because it is pure text-to-text retrieval. Mixing them would be wrong.

5.11 Evaluation metrics

Metric	What it measures	Where
Silhouette	Cluster separation (excluding noise)	`src/clustering.py`, `evaluate_system.py`
Adjusted Rand Index (ARI)	Agreement with ground truth, chance-corrected	`evaluate_system.py`
NMI / V-measure	Information-theoretic cluster agreement	`evaluate_system.py`
Spearman ρ	Rank correlation of held-out engagement, split by account	`scripts/select_models.py`, `scripts/evaluate_real.py`
hit@k / MRR	Does retrieval surface the human-curated cluster; at what rank	`src/retrieval.py`
CLIPScore	Cosine similarity between a generated caption and its image	`scripts/select_caption_model.py`
Trend precision@5	Fraction of top groups confirmed Rising by the definition	`scripts/evaluate_real.py`
Textual anonymity	Within-group hashtag overlap — lower means "unnamed" trends	`evaluation_against_popular_baselines.md`

Baselines compared against (`evaluation_against_popular_baselines.md`): hashtag/social-listening, keyword/Google-Trends-style frequency, and engagement-band trending lists — the tools people actually use — rather than alternate ML components. Current real-data headline: TrendLens coverage **0.822** vs hashtag **0.195**; held-out engagement ρ **0.291** vs keyword **−0.093**; textual anonymity **0.041** vs hashtag **0.332**.

6. Full pipeline walkthrough & artifacts

Commands (from `skills.md`)

```
source venv/bin/activate
cp .env.example .env            # set APIFY_API_TOKEN

python -m src.data_collector              # incremental (default)
python -m src.data_collector --baseline   # force full re-cluster
python -m src.data_collector --days 7     # override scan window

python -m src.rag "What cafe aesthetic is rising this week?"
python -m src.api                         # HTTP API server
python -m pytest tests/ -q                # test suite
```

Key data artifacts

Artifact	Produced by	Holds
`data/instagram/images/`	Phase 2	Downloaded post images
`data/instagram/posts.parquet`	Phase 1–2	Post metadata (real timestamps, likes, comments)
`data/instagram/embeddings.npy`	Phase 3 (CLIP)	512-d L2-normalised image vectors
`data/instagram/embed_meta.parquet`	Phase 3	Row-aligned metadata (row *i* matches embedding *i*)
`data/instagram/trends.json`	Phase 6	Per-theme summaries, growth, engagement, daily counts
`data/instagram/rag_index.faiss`	Phase 7 (bge)	FAISS inner-product index over RAG chunks
`data/instagram/rag_chunks.json`	Phase 7	The text chunks the index points to
`artifacts/cluster_registry.json`	Phase 4b	Stable cluster UUIDs, centroids, metadata
`artifacts/centroid_index.faiss`	Phase 4b	FAISS index over locked centroids

Three provenance guarantees to mention

- **Row alignment:** metadata row index equals embedding row index, asserted, not assumed (`assert_alignment`).
- **Checkpointed embeddings:** a disconnected runtime resumes from a checkpoint instead of recomputing everything.
- **Stale-artifact detection:** a sidecar manifest records which encoder produced the vectors, so a mismatched checkpoint is caught rather than silently reused.

7. Data integrity & key design decisions

The project's non-negotiable policy (from `skills.md`):

1. **No fabricated results.** Nothing reported that was not measured; unmeasured fields are null/omitted, never invented.
2. **Instagram data is real.** Timestamps, likes, comments, views and images come from public accounts via Apify.
3. **VLM output is interpretation, not ground truth.** Cluster names/descriptions come from BLIP captions.
4. **The LLM is a writing layer only.** Answers are formed from artifacts first.
5. **Answers hide pipeline internals.** No cluster IDs, scores, or lifecycle labels shown to users.

Design decisions worth quoting

#	Decision	Reason
1	Instagram via Apify	Real public post data, no scraping infrastructure
2	CLIP for embeddings	One joint space for images and text
3	UMAP-15 → HDBSCAN	Proven pipeline for visual groups; noise-aware
4	BLIP for captioning	CPU-capable VLM for cluster interpretation
5	Separate bge model for RAG	Text-to-text retrieval is a different task from image–text
6	Strict trend definition	"InsufficientData" is better than a confident wrong number
7	Stable cluster IDs + centroid locking	Time-series history survives re-runs
8	Idempotent pipeline	Re-running only adds new posts
9	LLM optional + constrained	The product works without it; it cannot fabricate
10	Centralised model config	One source of truth for every checkpoint name

8. Likely review questions & model answers

Short, confident answers you can give under pressure. Each names a concrete file or measured number.

Q. What is CLIP, in one line?

A dual-encoder network trained contrastively on ~400M image–caption pairs that projects images and text into one shared 512-d space where matching pairs have high cosine similarity.

Q. How does CLIP work internally?

A ViT-B/32 image tower cuts a 224×224 image into 32×32 patches (49 of them) plus a `[CLS]` token, runs them through 12 Transformer blocks, and projects the `[CLS]` output to 512-d. A separate text Transformer encodes the caption and projects it to the same 512-d space. Training makes the N matching pairs in each batch similar while pushing each away from the other N−1 captions. Inference is two forward passes and a dot product — it is not generative.

Q. Why CLIP and not train a classifier?

Visual trends are open-ended and unnamed; a classifier is limited to a fixed label set and needs retraining for each new category. CLIP gives an open vocabulary: I can describe a look in words at query time, and the same space powers clustering, retrieval and zero-shot tagging.

Q. Why UMAP before HDBSCAN?

Distances concentrate in 512 dimensions and degrade density clustering. UMAP first exposes the local manifold, then HDBSCAN finds groups. I compared raw-versus-UMAP and measured across 80 held-out account splits; 15-d UMAP was best (ρ 0.248 vs 0.170 at 10-d).

Q. Why HDBSCAN over KMeans or DBSCAN?

KMeans forces every point into k clusters and needs k in advance; visual trends have an unknown count and a real "none of the above". DBSCAN needs one global density threshold; HDBSCAN handles varying densities and, crucially, outputs noise as a first-class label (`-1`), so I never fabricate membership for outliers.

Q. What is the modality gap and how did you handle it?

CLIP image and text vectors sit in slightly separate regions, so raw image–text cosines occupy a narrow band and mostly reflect CLIP's geometry rather than the image. Without correction, the same two style tags won on 8 of 12 clusters. I subtract each image's own mean across tags before scoring, which makes the score "how much more this tag fits this image than the average" — and the clusters become separable.

Q. Why BLIP and not a newer multimodal LLM like GPT-4V?

Cost, control and reproducibility. BLIP runs locally on CPU, so the pipeline is self-contained and deterministic given seeds. I measured base versus large and chose large for lexical diversity and zero degenerate repeats, accepting ~0.04 img/s. Any API VLM could be plugged in because captioning is injected as a callable.

Q. Is the LLM hallucinating the advice?

No — structurally it cannot. It receives only a JSON bundle of measured facts, the shot steps are authored in code and tied to style tags CLIP actually measured, and the system prompt forbids inventing advice, hashtags, cameras or like counts. On any failure it returns `None` and a deterministic formatter answers instead.

Q. What is a "trend", formally, in your system?

A cluster is Rising only if it satisfies six gates at once — support (≥8 recent posts), persistence (≥3 active days), author breadth (≥3 accounts), growth margin (≥+50%/day), significance, and engagement coverage (≥60%) — where significance is an exact one-sided binomial test of recent-versus-prior rate. Otherwise it is "InsufficientData".

Q. Why is "InsufficientData" an acceptable answer?

Because the honest alternative on a single-snapshot corpus is a confident number built from noise. 87% of posts fall in one 11-day window, so a recent-vs-prior comparison is comparing a handful of posts. The strict definition makes the system say "I do not know" rather than mislead.

Q. How do you evaluate without labels?

Multiple ways: clustering with ground-truth blobs (silhouette, ARI, NMI, V-measure); retrieval with human-curated per-cluster labels (hit@k, MRR); encoder choice with held-out by-account engagement rank correlation (Spearman ρ); captions with CLIPScore; and the trend task against real-world baselines — hashtag, keyword and engagement-band detectors.

Q. How is this better than hashtag or keyword trend detection?

Measured on the real corpus: assignment coverage 0.822 vs 0.195 for hashtags; held-out engagement ρ 0.291 vs −0.093 for keyword frequency; and textual anonymity — visual groups share 8× fewer hashtags (0.041 vs 0.332), which is direct evidence the system finds visually-similar posts that have no shared name yet.

Q. What are the main limitations?

The corpus is small and heavy-tailed; a single snapshot cannot confirm longitudinal trends (hence the SQLite history store that will make claims testable as the collector runs daily); BLIP large is slow; and the historical evaluation used synthetic engagement labels, which the code labels loudly and never quotes as research findings.

Q. What is zero-shot here?

Tagging photography styles with text prompts that were never part of any training label set — the style taxonomy is written by hand and scored through CLIP's shared space, so no style-specific training was needed.

9. Cheat sheet & glossary

Numbers to have on the tip of your tongue

Thing	Value
CLIP embedding dimension	512
CLIP image input	224 × 224, patches 32 × 32 → 49 patches + [CLS]
CLIP image Transformer	12 layers, width 768
CLIP pretraining scale	~400M image–caption pairs
UMAP production dimensions	15 (from 512)
HDBSCAN production	min_cluster_size=5, min_samples=3, eom
FAISS index	IndexFlatIP (exact inner product = cosine)
Cluster KNN assignment threshold	0.25 cosine
Style taxonomy	15 tags, 5 aspects, 30 prompts; MIN_STYLE_SCORE=0.55
Trend windows	recent 4 days, prior 4 days
Scope gate thresholds	domain ≥ 0.22 and margin ≥ −0.01
RAG retrieval depth	k = 5
RAG text encoder	BAAI/bge-base-en-v1.5 (768-d)
Captioner	Salesforce/blip-image-captioning-large

Glossary

CLIP — Contrastive Language–Image Pre-training; dual-encoder model mapping images and text to one shared space.

Embedding — a fixed-length vector representing an input, positioned so that similar inputs are close together.

Cosine similarity — the cosine of the angle between two vectors; equals the dot product when both are unit length.

L2 normalisation — scaling a vector to length 1 so cosine similarity becomes a dot product.

Zero-shot — classifying or tagging using labels/prompts the model never trained on.

Modality gap — the systematic offset between image and text regions of CLIP's shared space.

ViT — Vision Transformer; treats an image as a sequence of patches processed by self-attention.

UMAP — a manifold-learning method for dimensionality reduction that preserves local neighbour structure.

HDBSCAN — hierarchical, density-based clustering with automatic cluster count and explicit noise.

FAISS — a library for fast nearest-neighbour search over vectors.

IndexFlatIP — FAISS's exact inner-product index; with unit vectors it performs cosine search.

BLIP — a vision-language model used here to caption images into text.

RAG — Retrieval-Augmented Generation; retrieve relevant evidence, then let a model write from it.

Modality / VLM (vision-language model) — a model that handles images and text together.

Centroid — the mean vector of a cluster's members; used as a stable anchor for assignment.

Poisson/binomial growth test — a significance test of whether a higher recent count is unlikely under a no-growth null.

MRR — Mean Reciprocal Rank; average of 1/rank of the first correct result.

ARI — Adjusted Rand Index; chance-corrected agreement between two clusterings.

CLIPScore — cosine similarity between a generated caption's text vector and the image vector.

> **End of document.** You can now explain CLIP internally, name every pretrained model and why it was chosen, and describe every algorithm in the pipeline with the measured evidence behind it.